

Documento de Requerimientos de Software (SRS) — Web App de Inventario y Registro de Artículos

Versión: 8.0 (Especificación de Arquitectura Final de Producción)

Estado: Aprobado para Desarrollo

Stack Objetivo: SvelteKit + TailwindCSS + Supabase Cloud (PostgreSQL, Auth, Storage, RLS) + Vitest

1. Visión General del Sistema

Sistema Web de Gestión de Inventario y Registro de Artículos (Caja Web Simplificada) optimizado para papelerías. El sistema permite administrar productos con ventas fraccionadas, auditar de forma automática e inmutable todo movimiento de almacén mediante triggers y RPCs, escanear códigos de barras USB con captura global de foco a nivel de ventana, y procesar ventas o devoluciones atómicas e idempotentes en PostgreSQL. La solución elimina por completo la eliminación física de registros (Soft Delete obligatorio) y bloquea accesos no autorizados mediante middleware en SvelteKit.

2. Requerimientos Funcionales

2.1. Catálogo, Aislamiento de Costos y Soft Delete

- **Campos públicos (Tabla products):** sku_code, name, description, price, stock (NUMERIC(10,3)), min_stock, image_url, is_active.
- **Aislamiento de costo (Tabla product_costs):** Relación 1:1 protegida por RLS exclusiva para el rol admin.
- **Prohibición de Eliminación Física (Soft Delete):** Se prohíbe la instrucción DELETE en products para evitar errores de llave foránea en tablas de historial. El borrado se ejecuta cambiando is_active = false.
- **RPC de Alta/Edición Atómica:** La creación o actualización de productos y sus costos se procesa mediante la RPC upsert_product_with_cost en una sola transacción.
- **Auditoría Automática por Triggers:** Cualquier modificación manual en el stock de la tabla products dispara automáticamente un TRIGGER AFTER UPDATE que inserta un registro inmutable en inventory_logs.

2.2. Frontend: Escáner y Middleware de Rutas

- **Captura Resiliente de Lectores USB:** SvelteKit implementa un window.addEventListener('keydown') global que intercepta ráfagas de teclado del lector de código de barras (<100ms entre caracteres) y redirige el buffer al carrito sin importar dónde esté el foco del cursor.
- **Guardias de Ruta en Servidor (hooks.server.ts):** El middleware de SvelteKit evalúa el

rol del JWT en cada petición SSR. Si un usuario con rol cajero intenta acceder a /admin/*, el servidor responde inmediatamente con una redirección 303 a /caja.

3. Control de Acceso y Roles (RBAC + RLS)

Módulo / Operación	Rol Administrador	Rol Cajero / Operador
Consultar Catálogo Activo (products)	Permitido	Permitido
Consultar Costos (product_costs)	Permitido	Denegado (Bloqueado por RLS)
Registrar Venta (RPC process_stock_outlet)	Permitido	Permitido
Anular Venta / Devolución (RPC cancel_stock_outlet)	Permitido	Denegado (Solo Admin)
Crear/Editar Productos con Costo (RPC upsert_product_with_cost)	Permitido	Denegado (Solo Admin)
Eliminar Producto Físicamente (DELETE)	Denegado (Usar Soft Delete)	Denegado
Acceso a Rutas Web /admin/* (SvelteKit Middleware)	Permitido	Redirigido a /caja

4. Esquema DDL + RPCs + Triggers v8.0 (Producción Final)

```
-- 1. ENUMS
CREATE TYPE user_role AS ENUM ('admin', 'cajero');
```

```
CREATE TYPE inventory_change_type AS ENUM ('VENTA', 'DEVOLUCION', 'REABASTECIMIENTO', 'AJUSTE_MANUAL', 'MERMA');
```

```
-- 2. TRIGGER DE AUTH BOOTSTRAP (Primer usuario = Admin)
```

```
CREATE OR REPLACE FUNCTION public.handle_first_user_admin()
```

```
RETURNS TRIGGER AS $$
```

```
BEGIN
```

```
IF (SELECT COUNT(*) FROM auth.users) = 0 THEN
```

```
NEW.raw_app_meta_data := COALESCE(NEW.raw_app_meta_data, '{}::jsonb') || '{"role": "admin"}::jsonb';
```

```
ELSE
```

```
NEW.raw_app_meta_data := COALESCE(NEW.raw_app_meta_data, '{}::jsonb') || '{"role": "cajero"}::jsonb';
```

```
END IF;
```

```
RETURN NEW;
```

```
END;
```

```
$$ LANGUAGE plpgsql SECURITY DEFINER SET search_path = public;
```

```
CREATE OR REPLACE TRIGGER on_auth_user_created_set_role
```

```
BEFORE INSERT ON auth.users
```

```
FOR EACH ROW EXECUTE FUNCTION public.handle_first_user_admin();
```

```
-- 3. TABLA DE PRODUCTOS
```

```
CREATE TABLE products (
```

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
sku_code VARCHAR(64) UNIQUE NOT NULL,
```

```
name VARCHAR(255) NOT NULL,
```

```
description TEXT,
```

```
price NUMERIC(10, 2) NOT NULL CHECK (price >= 0),
```

```
stock NUMERIC(10, 3) NOT NULL DEFAULT 0 CHECK (stock >= 0),
```

```
min_stock NUMERIC(10, 3) NOT NULL DEFAULT 5,
```

```
image_url TEXT,
```

```
is_active BOOLEAN NOT NULL DEFAULT true,
```

```
created_at TIMESTAMPTZ DEFAULT now(),
```

```
updated_at TIMESTAMPTZ DEFAULT now()
```

```
);
```

```
-- 4. TABLA DE COSTOS
```

```
CREATE TABLE product_costs (
```

```
product_id UUID PRIMARY KEY REFERENCES products(id) ON DELETE CASCADE,
```

```
cost NUMERIC(10, 2) NOT NULL CHECK (cost >= 0),
```

```
updated_at TIMESTAMPTZ DEFAULT now()
```

```
);
```

```
-- 5. CABECERA DE SALIDAS (Idempotente)
```

```
CREATE TABLE stock_outlets (
```

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
folio SERIAL UNIQUE,
```

```
user_id UUID NOT NULL REFERENCES auth.users(id),
```

```
total_amount NUMERIC(10, 2) NOT NULL CHECK (total_amount >= 0),
```

```
idempotency_key UUID UNIQUE,
```

```
is_canceled BOOLEAN NOT NULL DEFAULT false,
```

```
canceled_at TIMESTAMPTZ,
```

```
canceled_by UUID REFERENCES auth.users(id),
```

```
cancel_reason TEXT,
```

```
created_at TIMESTAMPTZ DEFAULT now()
```

```
);
```

```
-- 6. RENGLONES DE SALIDA
```

```
CREATE TABLE stock_outlet_items (
```

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
outlet_id UUID NOT NULL REFERENCES stock_outlets(id) ON DELETE CASCADE,
```

```
product_id UUID NOT NULL REFERENCES products(id),
```

```
quantity NUMERIC(10, 3) NOT NULL CHECK (quantity > 0),
```

```
unit_price NUMERIC(10, 2) NOT NULL,
```

```
subtotal NUMERIC(10, 2) NOT NULL
```

```
);
```

-- 7. TABLA DE AUDITORÍA INMUTABLE

```
CREATE TABLE inventory_logs (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  product_id UUID NOT NULL REFERENCES products(id),  
  change_type inventory_change_type NOT NULL,  
  previous_stock NUMERIC(10, 3) NOT NULL,  
  new_stock NUMERIC(10, 3) NOT NULL,  
  quantity_changed NUMERIC(10, 3) NOT NULL,  
  reference_id UUID,  
  created_by UUID REFERENCES auth.users(id),  
  notes TEXT,  
  created_at TIMESTAMPTZ DEFAULT now()  
);
```

-- 8. TRIGGER DE AUDITORÍA AUTOMÁTICA EN 'products'

```
CREATE OR REPLACE FUNCTION log_product_stock_changes()  
RETURNS TRIGGER AS $$  
BEGIN  
  IF OLD.stock IS DISTINCT FROM NEW.stock THEN  
    INSERT INTO inventory_logs (  
      product_id,  
      change_type,  
      previous_stock,  
      new_stock,  
      quantity_changed,  
      created_by,  
      notes  
    ) VALUES (  
      NEW.id,  
      CASE  
        WHEN NEW.stock > OLD.stock THEN 'REABASTECIMIENTO':inventory_change_type  
        ELSE 'AJUSTE_MANUAL':inventory_change_type  
      END,  
      OLD.stock,  
      NEW.stock,  
      NEW.stock - OLD.stock,  
      auth.uid(),  
      'Actualización manual de stock en catálogo'  
    );  
  END IF;  
  RETURN NEW;  
END;  
$$ LANGUAGE plpgsql SECURITY DEFINER SET search_path = public;
```

```
CREATE TRIGGER trg_audit_product_stock  
  AFTER UPDATE ON products  
  FOR EACH ROW EXECUTE FUNCTION log_product_stock_changes();
```

-- 9. CONFIGURACIÓN RLS STRICT (Sin políticas DELETE)

```
ALTER TABLE products ENABLE ROW LEVEL SECURITY;  
ALTER TABLE product_costs ENABLE ROW LEVEL SECURITY;  
ALTER TABLE stock_outlets ENABLE ROW LEVEL SECURITY;  
ALTER TABLE stock_outlet_items ENABLE ROW LEVEL SECURITY;  
ALTER TABLE inventory_logs ENABLE ROW LEVEL SECURITY;
```

```
CREATE POLICY "Lectura productos activos" ON products FOR SELECT TO authenticated USING (true);  
CREATE POLICY "Insert productos Admin" ON products FOR INSERT WITH CHECK ((auth.jwt() -> 'app_metadata' ->>  
'role') = 'admin');  
CREATE POLICY "Update productos Admin" ON products FOR UPDATE TO authenticated USING ((auth.jwt() ->  
'app_metadata' ->> 'role') = 'admin');
```

```
CREATE POLICY "Costos solo Admin" ON product_costs FOR ALL TO authenticated  
  USING ((auth.jwt() -> 'app_metadata' ->> 'role') = 'admin');
```

```
CREATE POLICY "Salidas propias o Admin" ON stock_outlets FOR SELECT TO authenticated  
  USING (auth.uid() = user_id OR (auth.jwt() -> 'app_metadata' ->> 'role') = 'admin');
```

```
CREATE POLICY "Logs solo Admin" ON inventory_logs FOR SELECT TO authenticated
USING ((auth.jwt() -> 'app_metadata' ->> 'role') = 'admin');
```

```
-- 10. RPC ATÓMICA: Crear / Editar Producto con Costo (Solo Admin)
```

```
CREATE OR REPLACE FUNCTION upsert_product_with_cost(
  p_id UUID,
  p_sku_code VARCHAR,
  p_name VARCHAR,
  p_description TEXT,
  p_price NUMERIC,
  p_cost NUMERIC,
  p_stock NUMERIC,
  p_min_stock NUMERIC,
  p_image_url TEXT
) RETURNS UUID AS $$
DECLARE
  v_product_id UUID := COALESCE(p_id, gen_random_uuid());
BEGIN
  IF (auth.jwt() -> 'app_metadata' ->> 'role') != 'admin' THEN
    RAISE EXCEPTION 'Solo un Administrador puede modificar el catálogo.';
  END IF;

  INSERT INTO products (id, sku_code, name, description, price, stock, min_stock, image_url, updated_at)
  VALUES (v_product_id, p_sku_code, p_name, p_description, p_price, p_stock, p_min_stock, p_image_url, NOW())
  ON CONFLICT (id) DO UPDATE SET
    sku_code = EXCLUDED.sku_code,
    name = EXCLUDED.name,
    description = EXCLUDED.description,
    price = EXCLUDED.price,
    stock = EXCLUDED.stock,
    min_stock = EXCLUDED.min_stock,
    image_url = EXCLUDED.image_url,
    updated_at = NOW();

  INSERT INTO product_costs (product_id, cost, updated_at)
  VALUES (v_product_id, p_cost, NOW())
  ON CONFLICT (product_id) DO UPDATE SET
    cost = EXCLUDED.cost,
    updated_at = NOW();

  RETURN v_product_id;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER SET search_path = public;
```

```
-- 11. RPC ATÓMICA E IDEMPOTENTE: Procesar Salida / Venta
```

```
CREATE OR REPLACE FUNCTION process_stock_outlet(
  p_items JSONB,
  p_idempotency_key UUID DEFAULT NULL
) RETURNS UUID AS $$
DECLARE
  v_outlet_id UUID;
  v_item JSONB;
  v_product_id UUID;
  v_qty NUMERIC(10, 3);
  v_db_price NUMERIC(10, 2);
  v_current_stock NUMERIC(10, 3);
  v_new_stock NUMERIC(10, 3);
  v_total NUMERIC(10, 2) := 0;
BEGIN
  IF p_idempotency_key IS NOT NULL THEN
    SELECT id INTO v_outlet_id FROM stock_outlets WHERE idempotency_key = p_idempotency_key;
```

```

IF FOUND THEN
    RETURN v_outlet_id;
END IF;
END IF;

```

```

INSERT INTO stock_outlets (user_id, total_amount, idempotency_key)
VALUES (auth.uid(), 0, p_idempotency_key)
RETURNING id INTO v_outlet_id;

```

```

FOR v_item IN
    SELECT value FROM jsonb_array_elements(p_items)
    ORDER BY (value->>'product_id')::UUID
LOOP
    v_product_id := (v_item->>'product_id')::UUID;
    v_qty := (v_item->>'quantity')::NUMERIC(10, 3);

```

```

SELECT stock, price INTO v_current_stock, v_db_price
FROM products WHERE id = v_product_id AND is_active = true
FOR UPDATE;

```

```

IF v_current_stock IS NULL THEN
    RAISE EXCEPTION 'Producto ID % inactivo o inexistente', v_product_id;
END IF;

```

```

IF v_current_stock < v_qty THEN
    RAISE EXCEPTION 'Stock insuficiente para ID %. Disponible: %, Solicitado: %',
        v_product_id, v_current_stock, v_qty;
END IF;

```

```

v_new_stock := v_current_stock - v_qty;

```

```

UPDATE products SET stock = v_new_stock, updated_at = NOW() WHERE id = v_product_id;

```

```

INSERT INTO stock_outlet_items (outlet_id, product_id, quantity, unit_price, subtotal)
VALUES (v_outlet_id, v_product_id, v_qty, v_db_price, v_qty * v_db_price);

```

```

INSERT INTO inventory_logs (product_id, change_type, previous_stock, new_stock, quantity_changed, reference_id,
created_by)
VALUES (v_product_id, 'VENTA', v_current_stock, v_new_stock, -v_qty, v_outlet_id, auth.uid());

```

```

v_total := v_total + (v_qty * v_db_price);
END LOOP;

```

```

UPDATE stock_outlets SET total_amount = v_total WHERE id = v_outlet_id;
RETURN v_outlet_id;
END;

```

```

$$ LANGUAGE plpgsql SECURITY DEFINER SET search_path = public;

```

```

-----
-- 12. RPC ATÓMICA: Cancelar Salida / Devolución (Solo Admin)
-----

```

```

CREATE OR REPLACE FUNCTION cancel_stock_outlet(
    p_outlet_id UUID,
    p_reason TEXT
) RETURNS VOID AS $$
DECLARE
    v_item RECORD;
    v_current_stock NUMERIC(10, 3);
    v_new_stock NUMERIC(10, 3);
    v_is_canceled BOOLEAN;
BEGIN
    IF (auth.jwt() -> 'app_metadata' ->> 'role') != 'admin' THEN
        RAISE EXCEPTION 'Solo un Administrador puede cancelar salidas.';
    END IF;

```

```

SELECT is_canceled INTO v_is_canceled FROM stock_outlets WHERE id = p_outlet_id FOR UPDATE;

```

```
IF v_is_canceled IS NULL OR v_is_canceled = true THEN
    RAISE EXCEPTION 'La salida no existe o ya fue cancelada previamente.';
END IF;
```

```
FOR v_item IN SELECT product_id, quantity FROM stock_outlet_items WHERE outlet_id = p_outlet_id LOOP
    SELECT stock INTO v_current_stock FROM products WHERE id = v_item.product_id FOR UPDATE;
    v_new_stock := v_current_stock + v_item.quantity;
```

```
UPDATE products SET stock = v_new_stock, updated_at = NOW() WHERE id = v_item.product_id;
```

```
INSERT INTO inventory_logs (product_id, change_type, previous_stock, new_stock, quantity_changed, reference_id,
created_by, notes)
VALUES (v_item.product_id, 'DEVOLUCION', v_current_stock, v_new_stock, v_item.quantity, p_outlet_id, auth.uid(),
p_reason);
END LOOP;
```

```
UPDATE stock_outlets
SET is_canceled = true, canceled_at = NOW(), canceled_by = auth.uid(), cancel_reason = p_reason
WHERE id = p_outlet_id;
END;
```

```
$$ LANGUAGE plpgsql SECURITY DEFINER SET search_path = public;
```